FC Fluxo de Caixa

DOCUMENTO 01

Contexto e requisitos

Requisitos funcionais e não funcionais refinados e análise de capacidade do pico

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

1. Problema de negócio 3

2. Premissas 3

3. Glossário (linguagem ubíqua) 4

4. Requisitos funcionais 4

5. Requisitos não funcionais 8

6. Restrições 9

7. Fora de escopo desta entrega 9

8. Análise do RNF crítico 9

9. Consequência para o desenho 12

FONTE: DOCUMENTOS-BRUTOS/01-CONTEXTO-E-REQUISITOS.MD

01 — Contexto e requisitos

1. Problema de negócio

Um comerciante precisa controlar o caixa do dia: cada entrada é um crédito, cada saída é um débito, e no fim do dia (ou a qualquer momento) ele precisa ver o saldo daquele dia e o saldo acumulado até a data. O valor para o negócio está em duas capacidades distintas:

1. **Não perder um lançamento.** O registro do movimento é o fato contábil. Se o relatório cair, a venda ainda tem de ser lançada.
2. **Enxergar o caixa consolidado.** O saldo diário é uma leitura derivada. Pode atrasar alguns segundos. Não pode contradizer, no estado estável, a soma dos lançamentos.

O enunciado separa isso em dois serviços e fixa o requisito que orienta a arquitetura: o controle de lançamentos permanece disponível se o consolidado diário cair; em pico, o consolidado atende **50 requisições por segundo** com **no máximo 5% de perda**.

2. Premissas

Premissas explícitas, porque o enunciado não fixa volume de escrita, número de comerciantes nem moeda.

ID	Premissa	Uso
P1	Moeda única: BRL, duas casas decimais.	Modelo `Dinheiro` e evento
P2	Um access token representa exatamente um comerciante. O cadastro do comerciante vive no provedor de identidade, não neste sistema.	Multi-tenant por claim
P3	`dataLancamento` é data de competência (`LocalDate`), informada pelo cliente. Pode ser retroativa. `criadoEm` é o instante do registro (UTC).	Estorno corrige o dia original
P4	Pico de leitura do consolidado = 50 req/s, sustentado pelo menos durante a janela de fechamento.	Dimensionamento
P5	Pico de escrita = 20 lançamentos/s. O enunciado não define esse número. Vinte por segundo cobre ~72 mil lançamentos numa hora de pico e é a premissa de outbox, SQS e banco de escrita. Abaixo disso, a folga aumenta.	Capacidade da escrita
P6	Hit ratio de cache no pico ≥ 80%. O dia corrente e a semana recente concentram as consultas.	Pool JDBC e SLO de latência
P7	Há um operador (ou integração) por comerciante. Não há, nesta versão, papel de contador enxergando vários comerciantes.	Fora de escopo de autorização
P8	Consistência do saldo é eventual, com meta p95 < 5 s entre o commit do lançamento e o reflexo na leitura.	SLO do blueprint

ID	Premissa	Uso
P9	Ambiente diário de desenvolvimento é local (Docker). A conta AWS tem `dev`, `hml` e `prod`; dev não é o lugar onde se codifica o dia.	Custo, ADR 0029
P10	Região: `us-east-1`. Transferência internacional com salvaguarda (cláusulas-padrão e DPA da AWS). Residência no Brasil volta a ser requisito só por contrato ou norma, com o custo do documento 08.	LGPD art. 33, ADR 0021

3. Glossário (linguagem ubíqua)

Termo	Significado neste sistema
Comerciante	Titular do caixa. Identificado pelo claim `comerciante_id` (UUID). Nunca é aceito no corpo ou na URL.
Lançamento	Fato imutável de crédito ou débito. Uma vez persistido, não se altera nem se apaga.
Crédito	Entrada de dinheiro. Aumenta o saldo do dia de competência.
Débito	Saída de dinheiro. Reduz o saldo do dia de competência.
Valor	Quantia positiva, em BRL, com no máximo duas casas. Zero e negativo são rejeitados.
Data de competência	Dia a que o movimento pertence (`dataLancamento`). É a chave do saldo diário.
Data de registro	Instante em que o sistema aceitou o lançamento (`criadoEm`).
Estorno	Novo lançamento, de tipo oposto, mesmo valor e mesma data de competência, apontando para o lançamento original. É a única correção admitida.
Saldo diário	`totalCreditos − totalDebitos` de um comerciante em uma data de competência.
Saldo acumulado	Soma dos saldos diários do comerciante em todas as datas `≤` a data consultada.
Consolidação	Aplicação do evento `LancamentoRegistrado` na projeção de saldo. Comutativa: a ordem de chegada não altera o resultado.
Consistência eventual	A leitura do consolidado pode não conter um lançamento recém-aceito. O atraso alvo é p95 < 5 s. O lançamento em si já é durável.
Chave de idempotência	Header `Idempotency-Key` (UUID) no POST. A mesma chave, do mesmo comerciante, com o mesmo corpo, devolve o mesmo recurso.
Outbox	Tabela `outbox_eventos` gravada na mesma transação do lançamento. O relay publica no SNS depois do commit.
Perda de requisição	Resposta de erro (5xx ou timeout visto pelo cliente) sobre o total de requisições HTTP ao consolidado. Não significa descarte de lançamento.

4. Requisitos funcionais

Critérios no formato Given/When/Then. O `comercianteId` dos exemplos é o do realm local: `11111111-1111-1111-1111-111111111111`.

RF-01 — Registrar crédito

Dado um token com escopo `lancamentos:write` e claim `comerciante_id`
Quando `POST /api/v1/lancamentos` com `tipo=CREDITO`, `valor=150.00`, `dataLancamento` válida, `descricao` de 1 a 255 caracteres e `categoria` opcional até 60
Então o sistema persiste o lançamento e o evento na mesma transação, responde **201** com o recurso (sem `comercianteId` vindo do cliente) e o saldo daquele dia passa a incluir 150,00 depois da consolidação.

RF-02 — Registrar débito

Dado o mesmo contexto de escrita
Quando o tipo é `DEBITO`
Então o comportamento é o do RF-01, e a consolidação soma o valor em `totalDebitos`.

RF-03 — Rejeitar valor inválido

Dado um token de escrita
Quando `valor` é zero, negativo, tem mais de duas casas, ou não é numérico; ou `descricao` é vazia ou maior que 255; ou `tipo` não é `CREDITO` nem `DEBITO`
Então nada é persistido, a resposta é **400** `application/problem+json` (RFC 9457) e nenhum evento é publicado.

RF-04 — Idempotência do POST

Dado um `Idempotency-Key` UUID ainda não usado por aquele comerciante
Quando o mesmo POST é repetido com a mesma chave e o mesmo corpo
Então existe um único lançamento; a primeira resposta é **201** e a repetição é **200** com o mesmo corpo.

Dado a mesma chave
Quando o corpo é diferente
Então a resposta é **409** e o lançamento original permanece.

Sem o header, cada POST é um lançamento novo. A chave é opcional, como no blueprint.

RF-05 — Consultar um lançamento

Dado um token com `lancamentos:read`
Quando `GET /api/v1/lancamentos/{id}` e o id pertence ao comerciante do token
Então a resposta é **200** com tipo, valor, moeda `BRL`, data, descrição, categoria, `lancamentoEstornadoId` e `criadoEm`.

Quando o id não existe ou pertence a outro comerciante
Então a resposta é **404** nos dois casos (não se revela existência cruzada).

RF-06 — Listar lançamentos do dia

Dado um token de leitura
Quando `GET /api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`
Então a resposta traz somente lançamentos daquele comerciante e daquela data de competência, paginados. `size` máximo efetivo: 50.

RF-07 — Estornar

Dado um lançamento do comerciante que ainda não foi estornado e que não é ele próprio um estorno
Quando `POST /api/v1/lancamentos/{id}/estornos` com escopo `lancamentos:write` e corpo opcional `{ "motivo": "..." }`
Então nasce um novo lançamento de tipo oposto, mesmo valor, mesma `dataLancamento`, `lancamentoEstornadoId` preenchido, e um evento `LancamentoRegistrado` v1. O original não é alterado.

Lacuna do blueprint, resolvida aqui e registrada no ADR 0015: o agregado não tem campo `motivo`, e o POST de estorno não envia `descricao`. A descrição do novo lançamento é o `motivo` (até 255 caracteres) ou, na ausência, `Estorno do lançamento {id}`. O motivo não entra no evento — o consolidado só precisa do tipo oposto, do valor e da data.

RF-08 — Estorno único

Dado um lançamento já estornado
Quando chega outro POST de estorno
Então a resposta é **409** e não nasce segundo estorno.

RF-09 — Estorno de estorno é proibido

Dado um lançamento cujo `lancamentoEstornadoId` não é nulo
Quando chega POST de estorno para ele
Então a resposta é **409**. A correção de um estorno errado é um lançamento novo, deliberado, não um estorno do estorno.

RF-10 — Lançamento sobrevive à queda do consolidado

Dado o `consolidado-service` indisponível, a fila acumulando e o banco do consolidado fora
Quando o comerciante registra um lançamento
Então o POST responde 201 no tempo do SLO de escrita, o registro está commitado em `lancamentos` e em `outbox_eventos`, e nenhuma chamada síncrona ao consolidado ocorre.

RF-11 — Saldo do dia

Dado um token com `consolidado:read`
Quando `GET /api/v1/consolidados/diario/{data}`
Então a resposta é **200** com `data`, `totalCreditos`, `totalDebitos`, `saldo`, `saldoAcumulado`, `quantidadeLancamentos` e `atualizadoEm`.

RF-12 — Dia sem movimento

Dado nenhuma projeção para aquela data
Quando o GET do dia é feito
Então a resposta é **200** com zeros, não 404. `saldoAcumulado` continua sendo a soma dos dias anteriores.

RF-13 — Relatório de período

Dado `inicio` e `fim` com `fim − inicio ≤ 366 dias`
Quando `GET /api/v1/consolidados/diario?inicio=&fim=`
Então a resposta lista cada dia do intervalo (inclusive dias zerados) e o saldo acumulado coerente com a

soma.

RF-14 — Período excessivo

Quando o intervalo supera 366 dias ou `inicio` > `fim`
Então a resposta é **400** e o banco não executa a consulta larga.

RF-15 — Isolamento entre comerciantes

Dado dois tokens com `comerciante_id` distintos
Quando um consulta id, lista, saldo ou estorno do outro
Então o resultado é o mesmo de "não existe": 404 no recurso, lista vazia, saldo zerado. Não há parâmetro de tenant na URL.

RF-16 — Escopos

Quando falta o escopo da operação (`lancamentos:write` , `lancamentos:read` ou `consolidado:read`)
Então a resposta é **403**. Token ausente ou inválido: **401**.

Na AWS a autoridade é `SCOPE_fluxo-caixa/lancamentos:write` (prefixo `fluxo-caixa/`). No Keycloak local o prefixo é vazio: `SCOPE_lancamentos:write` .

RF-17 — Consolidação do evento

Dado um `LancamentoRegistrado` v1 novo
Quando o consumidor processa
Então `totalCreditos` ou `totalDebitos` , `saldo` , `quantidadeLancamentos` e `atualizadoEm` mudam na linha `(comercianteId, dataLancamento)` , e `eventos_processados` guarda o `eventId` na mesma transação.

RF-18 — Estorno corrige o dia original

Dado um crédito de 150,00 em 2026-11-10 já consolidado
Quando o estorno é consolidado
Então o dia 2026-11-10 recebe um débito de 150,00. O saldo daquele dia volta ao valor anterior. Dias posteriores, no acumulado, acompanham essa correção. O dia "de hoje" não absorve o estorno se a competência era outra.

RF-19 — Reprocessamento não duplica saldo

Dado um `eventId` já presente em `eventos_processados`
Quando o SQS reentrega a mensagem (at-least-once)
Então o saldo não muda e a mensagem é removida da fila.

RF-20 — Erro no formato problema

Quando qualquer rejeição da API ocorre
Então o corpo é `application/problem+json` com `type` , `title` , `status` , `detail` e `instance` . Falha de domínio não vaza stack trace.

5. Requisitos não funcionais

Metas herdadas do blueprint e tornadas mensuráveis. O detalhamento de SLI, error budget e testes está no documento 05.

ID	Atributo	Meta	Como se mede
RNF-01	Disponibilidade de lançamentos	99,9% ao mês, com o consolidado no ar ou caído	Probes de readiness no ALB; minutos de 5xx/timeout no POST e GET de lançamentos
RNF-02	Disponibilidade do consolidado	99,5% ao mês	Idem para as rotas `/api/v1/consolidados*`
RNF-03	Latência de escrita	p95 < 200 ms, p99 < 500 ms	Histograma HTTP do POST, server-side
RNF-04	Latência de leitura	p95 < 100 ms com cache; p99 < 300 ms	Histograma do GET diário
RNF-05	Throughput e perda	≥ 50 req/s sustentados no consolidado; erro ≤ 5%; meta interna ≤ 1%	k6 `http_req_failed` e `http_reqs`
RNF-06	Atraso de consolidação	p95 < 5 s do commit até o saldo refletir	Diferença `atualizadoEm` / timestamp do evento vs. `criadoEm`
RNF-07	Durabilidade do lançamento	Zero perda após 201	Commit único lançamento+outbox; PITR do RDS
RNF-08	Retenção do buffer	Evento espera a volta do consumidor por até 14 dias	Retenção da fila `consolidado-lancamentos-queue`
RNF-09	RPO / RTO	RPO ≤ 5 min; RTO ≤ 1 h	PITR do RDS; runbook de restauração
RNF-10	Isolamento de falha	Falha do consolidado não aumenta o erro de lançamentos	Teste `tests/e2e/resiliencia.sh`
RNF-11	Segurança	OAuth2/OIDC, escopo, tenant no token, TLS, criptografia em repouso	06
RNF-12	Observabilidade	Trace único atravessando HTTP, outbox e SQS; alarme de DLQ e de lag	07
RNF-13	Manutenibilidade	Domínio sem framework; ArchUnit; API `/api/v1`; evento com `eventVersion`	ADR 0008, ADR 0017
RNF-14	Custo	Produção enxuta para o volume real, região `us-east-1`	08: cerca de USD 778/mês em prod
RNF-15	LGPD	Transferência internacional com salvaguarda, minimização, trilha imutável	ADR 0021
RNF-16	Escalabilidade	Horizontal, no mínimo 2 tasks em AZs distintas, auto scaling por CPU e requisições	05

Error budget mensal (30 dias = 43.200 min):

- Lançamentos, 0,1% → **43,2 minutos**.
- Consolidado, 0,5% → **216 minutos**.

"Perda de 5%" não é o error budget mensal. É o teto de erro **durante o pico de 50 req/s**. São coisas diferentes: o serviço pode estar dentro do SLO mensal e ainda assim reprovar o pico, ou o inverso.

6. Restrições

- Dois processos de deploy independentes: `lancamentos-service` (porta 8081) e `consolidado-service` (porta 8082).
- PostgreSQL 17, um banco por serviço. Em produção, duas instâncias RDS. Migração por Flyway.
- Integração somente pelo evento `LancamentoRegistrado` versão 1, tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq` (`maxReceiveCount` 5).
- Java 25 e Spring Boot 4. Cliente AWS SDK v2, sem Spring Cloud AWS.
- JWT validado em cada serviço. Cognito na AWS, Keycloak no local.
- Sem update e sem delete de lançamento.
- `valor` no evento é string decimal com duas casas.

7. Fora de escopo desta entrega

- Interface de usuário. O contrato é a API HTTP.
- Cadastro, convite e recuperação de senha do comerciante (ficam no Cognito/Keycloak).
- Múltiplas moedas, contas bancárias, PIX, boleto, conciliação e Open Finance.
- Fechamento de período, trava contábil e reabertura.
- Papel multi-comerciante (contador, franqueadora).
- Event sourcing como persistência do lançamento. O estado atual é uma linha; o evento é de integração.
- Active-active em duas regiões.
- BFF e API Gateway. A borda desta versão é CloudFront, WAF e ALB interno (ADR 0022).

Esses itens estão priorizados em evoluções futuras.

8. Análise do RNF crítico

Há duas obrigações, e elas não se resolvem com o mesmo mecanismo.

Obrigações. (1) Disponibilidade de lançamentos independente do consolidado. (2) O consolidado absorve 50 req/s com no máximo 5% de respostas perdidas.

Interpretação de "perda". O enunciado fala em perda de *requisições* ao serviço de consolidado. Isso é taxa de erro HTTP (5xx, timeout, conexão recusada) ≤ 5% no pico. Meta interna do blueprint: ≤ 1%. Lançamento aceito com 201 não é "perdido" se o saldo atrasar: ele está no Postgres e o evento espera na fila por até 14 dias. Tratar atraso de projeção como perda contábil seria o desenho errado.

8.1 Por que o lançamento não cai junto

Não existe chamada síncrona entre os serviços. O POST grava lançamento e outbox na mesma transação local e responde. O relay publica no SNS em outro ciclo. Se o SNS estiver fora, o 201 já foi dado e a linha permanece com `publicado_em` nulo. Se o consolidado, o Redis ou o RDS dele caírem, a fila segura a mensagem. Bancos, tasks ECS, target groups e auto scaling são separados. O teste que evidencia isso é `make resiliencia`.

O custo desse isolamento é consistência eventual e um componente a mais (relay + fila). O custo de não isolá-lo seria o caixa parar toda vez que o relatório falha — exatamente o que o enunciado proíbe.

8.2 Lei de Little no caminho de leitura

Notação: λ taxa de chegada, S tempo médio de serviço dentro do `consolidado-service`, L = λ × S número médio de requisições em voo.

Símbolo	Valor	Origem
λ	50 s^{-1}	RNF do enunciado
S com cache (S_hit)	0,040 s	JWT com JWKS em cache + GET no Redis + serialização. Premissa a confirmar no k6
S sem cache (S_miss)	0,120 s	Miss + duas queries indexadas (linha do dia e `SUM` do acumulado) + SET
Hit ratio h	0,80	Premissa P6

Tempo médio:

```
S = 0,80 × 0,040 + 0,20 × 0,120 = 0,056 s
L = 50 × 0,056 = 2,8 requisições em voo
```

Um burst de 3× (150 req/s) por alguns segundos, com S estável:

```
L_burst = 150 × 0,056 = 8,4 requisições em voo
```

O caminho é limitado por I/O, não por thread. Com virtual threads (Java 25), um pool de plataforma de 200 threads não é o teto. O risco de perder requisição está em task de menos durante um scale-out lento, em stampede de cache, em rate limit do WAF calibrado abaixo do pico e em timeout — não em “acabou a thread”.

8.3 Quantas tasks

Meta de capacidade, a ser comprovada no k6 antes de virar número de produção: **uma task de 1 vCPU / 2 GiB** (aplicação + sidecar ADOT) sustenta **150 req/s** com erro < 1% e p95 < 100 ms quando h ≥ 0,80.

Utilização ρ = λ / (n × μ), com μ = 150 req/s por task:

Situação	n	ρ
Pico nominal, mínimo de HA	2	50 / 300 = **17%**
Uma AZ ou uma task perdida (N−1)	1	50 / 150 = **33%**

Situação	n	ρ
Teto do auto scaling do consolidado	10	50 / 1.500 = 3% no pico de requisito. O Terraform para em 10 tasks, não em 6

Com ρ = 0,33 e variação moderada, a fila interna ainda não está no joelho (o crescimento áspero de latência aparece perto de 70%). Duas tasks em AZs distintas atendem o requisito com folga de N−1. O mínimo de duas tasks existe por disponibilidade, não porque uma task seja incapaz de 50 req/s.

Auto scaling: desired 2, mínimo 2, máximo 6, por CPU (alvo 60%) e `ALBRequestCountPerTarget`. O scale-out precisa começar antes de ρ = 0,70. Como o pico de 50 req/s já nasce confortável em 2 tasks, o scaling cobre crescimento e a perda de uma task, não o requisito nominal.

8.4 Conexões de banco

No miss há duas sentenças: a linha `(comerciante_id, data)` e o `SUM(saldo)` até a data. Índice único em `(comerciante_id, data)`. Um ano de caixa é ~365 linhas por comerciante; o `SUM` é um range scan curto. Por isso a projeção **não** materializa saldo acumulado em todas as linhas futuras — um estorno antigo reescreveria o ano inteiro.

```
λ_stmt = 50 × (1 − 0,80) × 2 = 20 sentenças/s
S_stmt = 0,008 s
L_db   = 20 × 0,008 = 0,16 conexões ativas
```

Pior caso desenhado (cache inutilizado, h = 0, query de acumulado em 20 ms, ainda em 2 tasks):

```
λ_stmt = 50 × 2 = 100 sentenças/s
L_db   = 100 × 0,020 = 2 conexões ativas
```

Pool Hikari por task: `minimumIdle = 2`, `maximumPoolSize = 10`.

Tasks do consolidado	Conexões reservadas
2 (mínimo)	20
6 (teto)	60

`max_connections` aproximado do RDS, fórmula `DBInstanceClassMemory / 9.531.392`:

Classe	RAM	Conexões típicas
`db.t4g.micro`	1 GiB	~100
`db.t4g.small`	2 GiB	~225
`db.t4g.medium`	4 GiB	~450

Sessenta conexões cabem num `db.t4g.small` com margem para o operador, o Flyway e o monitoramento. A classe medium seria folga de memória, não exigência do pico. A estimativa de custo usa **small Multi-AZ** por esse cálculo; medium está na sensibilidade do documento de custos.

O mesmo raciocínio na escrita, com a premissa P5:

```
λ = 20 s⁻¹
S_tx = 0,030 s   (INSERT do lançamento + INSERT da outbox + commit)
L = 20 × 0,030 = 0,6 conexões ativas
```

Pool de 10 em 2 tasks de lançamentos reserva 20 conexões. O banco de escrita também cabe em `db.t4g.small`.

8.5 Outbox, fila e a hora em que o consolidado fica fora

Relay: a cada 500 ms, lote de 100, `SELECT … FOR UPDATE SKIP LOCKED`. Uma réplica publica até 100 / 0,5 = **200 eventos/s**. Duas réplicas, cerca de 400 eventos/s, vinte vezes o pico de escrita da premissa P5. O `SKIP LOCKED` é o que permite duas tasks sem publicar o mesmo registro.

Se o consolidado ficar 1 hora fora, no pico de escrita:

```
20 evt/s × 3.600 s = 72.000 mensagens na fila
```

SQS Standard absorve isso sem teto prático nessa ordem de grandeza. Na volta, se cada task aplica ~100 eventos/s, duas tasks drenam 72 mil mensagens em **6 minutos**. A retenção de 14 dias, no mesmo pico sustentado (um teto irreal, útil só como limite), guarda 20 × 86.400 × 14 ≈ 24 milhões de mensagens. O alarme de idade da mensagem mais antiga dispara muito antes disso (ver observabilidade).

Orçamento de erro HTTP no pico, para não confundir com a fila:

```
50 × 0,05 = 2,5 respostas com falha por segundo  (teto do enunciado)
50 × 0,01 = 0,5 respostas com falha por segundo  (meta interna)
```

Numa hora de pico o enunciado admitiria 9.000 erros; a meta interna, 1.800. O threshold do k6 trava a meta interna (`http_req_failed: rate<0.01`), não o teto frouxo de 5%.

8.6 O que faria a solução reprovar o RNF

- Chamar o consolidado no meio do POST (acoplamento síncrono).
- Dual-write: commit no Postgres e publish no SNS sem outbox. Falha entre os dois perde o evento ou exige 201 sem garantia.
- Uma única task, ou as duas tasks agendadas na mesma AZ.
- Rate-based rule do WAF abaixo de 50 req/s por cliente legítimo que concentre o fechamento.
- Pool JDBC grande o bastante para derrubar o `max_connections` num stampede, se alguém “proteger” o banco abrindo 100 conexões por task.
- Tratar fila cheia como erro do POST. A fila cheia é problema do consumidor; o POST já terminou.

9. Consequência para o desenho

A arquitetura que cai fora dessa conta é a de dois serviços com bancos separados, integração por outbox + SNS/SQS, leitura em projeção com cache, e duas tasks por serviço. O restante dos documentos descreve esse desenho sem reabrir a conta, exceto quando um trade-off muda capacidade ou custo.